

Complexity

Complexity

Complex systems are simple parts running local rules whose collective behaviour is more than the sum of its parts.

A **complex system** is one whose collective behaviour cannot be straightforwardly derived from the parts in isolation.

Three principles recur across examples we will look at:

Simple units with short-range relationships.

Simple units operating in parallel, with no central controller.

*The whole exhibits **emergent** phenomena, patterns the parts do not contain.*

Emergence

Steven Johnson's five rules for designing a system in which macro-level intelligence derives from local knowledge:

More is different.

Only the entire system at work reveals global behaviour.

Ignorance is useful.

Densely interconnect simple elements; let sophistication trickle up.

Encourage random encounters.

Chance contact lets the macrostate be altered.

Look for patterns in the signs.

Pattern detection lets meta-information circulate.

Pay attention to your neighbours.

Local information becomes global wisdom.

The mapping problem

Every algorithmic **generator** (a cellular automaton, an attractor, a genetic search, a network) produces a stream of numbers. The **mapping** is the act of assigning those numbers to the parameters of sound: pitch, amplitude, timbre, position.

Given an N-dimensional state vector, how do we **assign** it to the M-dimensional space of sound such that the algorithm's **identity** survives the transit into perception?

Complexity

Complexity

Levy-Leblond (as cited in Vaggione, 2010) mentions that complexity can be understood to happen where ***mutual interactions*** between ***different levels*** are manifest. Koenig adds:

"Relationships can be produced or perceived within a parameter [...] Here we find a point for point dependence, such as the longer the softer, or a combination of clearly marked parameter/time fields, such as rising pitch and volume fluctuating around a central point. We shall speak of higher complexity if the individual values have many relationships to one another, and of low complexity if only a few relationships are clear."

(Koenig, 1965, p. 6)

Complexity

Koenig also introduced the concepts of ***direct*** and ***indirect*** complexity.

Direct as being when ***two*** clear distinguishable, successive or partially overlapping sounds are compared.

Indirect when ***several sounds*** forming a section are compared.

These relationships vary and the complexity of that variation is registered on the vertical axis.

Complexity

Complexity can be understood as occurring in sounds, their *relationship* through *parameter values* and the *variation* of these relationships.

Complexity can easily be explored through different *parametrical configurations*.

Complexity

“A new systems model in music and its processes will surely redefine the role of the composer or artist who until now has been seen as the mastermind of precisely controllable variables. The new paradigm will more likely see that person’s role as a guiding element in a complex process that links artifact to context.”

(Truax, 1992, p. 30)

Complex systems

A complex system is typically defined as a system that is “*more than the sum of its parts.*”

The individual elements of the system may be incredibly simple and easily understood, the behaviour of the system as a whole can be *highly complex, intelligent*, and *difficult to predict*.

Complex systems

There are ***three*** main modern approaches:

- (1) How ***interactions*** give rise to patterns of behavior,
- (2) Understanding the ways of ***describing*** complex systems
- (3) The ***process of formation*** of complex systems through pattern formation and evolution.

Complex systems

Considerations of complex systems. (*The Nature of Code*)

Non-linearity. A small change in initial conditions can have a massive effect on the outcome. Non-linear systems are a superset of chaotic systems. Small changes can have huge results.

Competition and cooperation. Elements can “cooperate”— i.e. work together to stay together and move together. ***Separation***, however, will ask the elements to “compete” for space.

Feedback. Complex systems often include a feedback loop where the the output of the system is fed back into the system to influence its behaviour in a positive or negative direction.

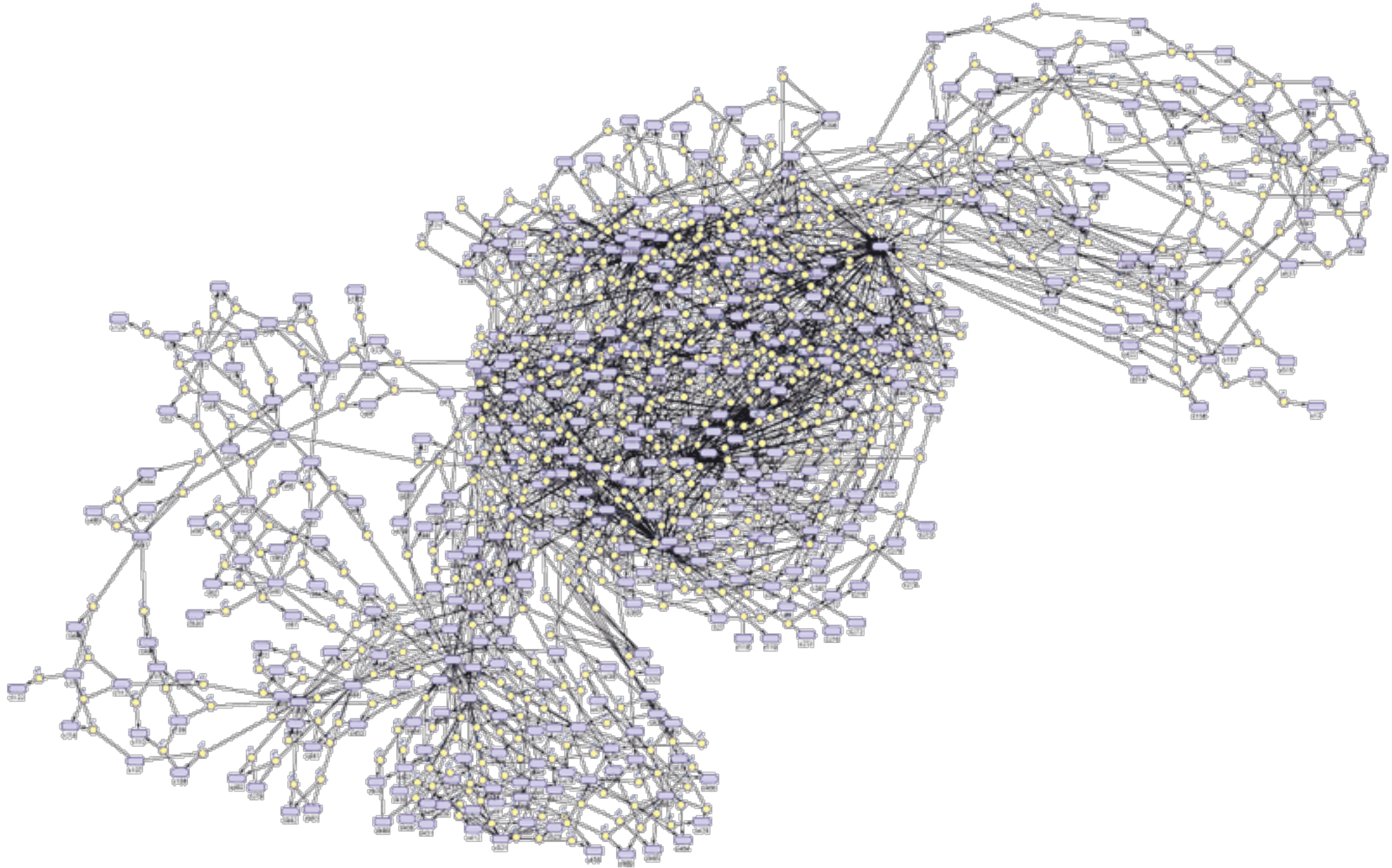
Complex systems

Three key principles of complex systems.

- * Simple units with ***short-range relationships***.
- * Simple units operate in ***parallel***.
- * System as a whole exhibits ***emergent phenomena***. Out of the interactions between these simple units emerges complex behavior, patterns, and intelligence.

(The Nature of Code, (Shiffman, 2012))

Complex systems



(From '<http://www.mathworks.com/>')

Emergence

How low-level rules can create higher-level sophisticated structures.

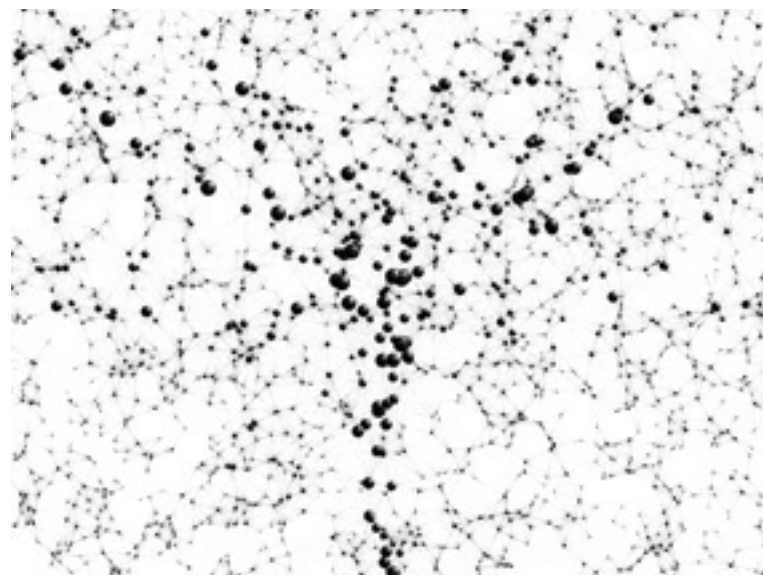
Emergence is defined by the way complex systems and patterns arise out of a multiplicity of relatively simple interactions. It is the “*case where a joint effect of several causes cannot be reduced, or traced back, to its component causes*” (Whitlaw)

Emergence remains a slippery term and its application to music is not always obvious. “*Different accounts often disagree on whether an entity is emergent; and when they agree, there is often no clear basis for this agreement.*” (Wilson, 2010.) The problematic and ‘*magical getting something from nothing*’.

Emergence

Mark Bedau defines two main principles of emergence:

- (1) Emergent phenomena are somehow ***constituted by***, and ***generated from***, underlying processes.
- (2) Emergent phenomena are somehow ***autonomous*** from underlying processes.



Emergence

Emergence can be subdivided into two perspectives, that of **weak emergence** and **strong emergence**.

Strong emergence is when the whole is really greater than the sum of its parts and when the “*truths concerning that phenomenon are not deducible even in principle from truths in the low-level domain.*”
(Chalmers, 2006)

Weak emergence leads to novel properties in systems as the result of interactions at elemental levels where the “*truths concerning that phenomenon are unexpected given the principles governing the low-level domain.*”

Emergence

How low-level rules can create higher-level sophisticated structures.

- * **Observe** the system as whole at work. Focus on global behaviour but design the lower level activity.
- * Keep elemental units **simple** but sophisticated behaviour come from their **interaction**.
- * Encourage **random encounters** among entities for generating surprises and shift in focus.
- * Observe your **neighbours**. Local interaction is the most important.

S. Johnson (2001, p. 78)

Emergence

Emergence is often said to occur from very *simple micro-level operations*. It is also frequently used in relation to adaptive or *self-organizing* systems.

In phenomena occurring from weak emergence it is not required to possess these qualities in order qualify as emergent. Emergence can indeed be understood as coming purely from interaction.

“Computational emergence arises through the interaction of elements, regardless of their relationship to the observer's model”
(*Whitlaw*, 1998).

Emergence

“The conceptual separation of composition (or, responsibility on premises and conditions, in my wording) and music (sonic features arising from premises and conditions) perhaps attests to a shared perspective. The implication is that sound is the epiphenomenon of a lower-level process: you design a low-level process, and the interactions and interferences among particle components taking part in the process are heard as a dynamic shape of sound, a process of sonological emergence.”

(Di Scipio, 2005, p. 18)

Emergence

“A musical composition constitutes an emergence because it is not reducible to its parts or elements or even to the operations that were realized during its composition. Therefore we can neglect here the word << Causality >>. [...] The musical work will constitute itself as an emergence (a singularity), and not as a simple globality, since it manifests itself not as that which constitutes the product from the parts of a whole, or even as the epiphenomenon of its components, but as simply as that what it is, without any reduction.”

(Vaggione, 2010, p.55)

Agents

Artificial Agents

A possible path of generative music is to compose ***autonomous artificial universes*** that can generate original aesthetic experiences.

Compositions can generate and self-organize, shift and oscillate, adapt, develop, and becoming whole: like an organism.

Generative processes can be designed so that they exhibit a ***range of behaviors*** within ***constraints*** arising from the underlying, artist-devised mechanisms.

To create a system, or process, that is set in motion and runs with some degree of ***autonomy***.

Agents

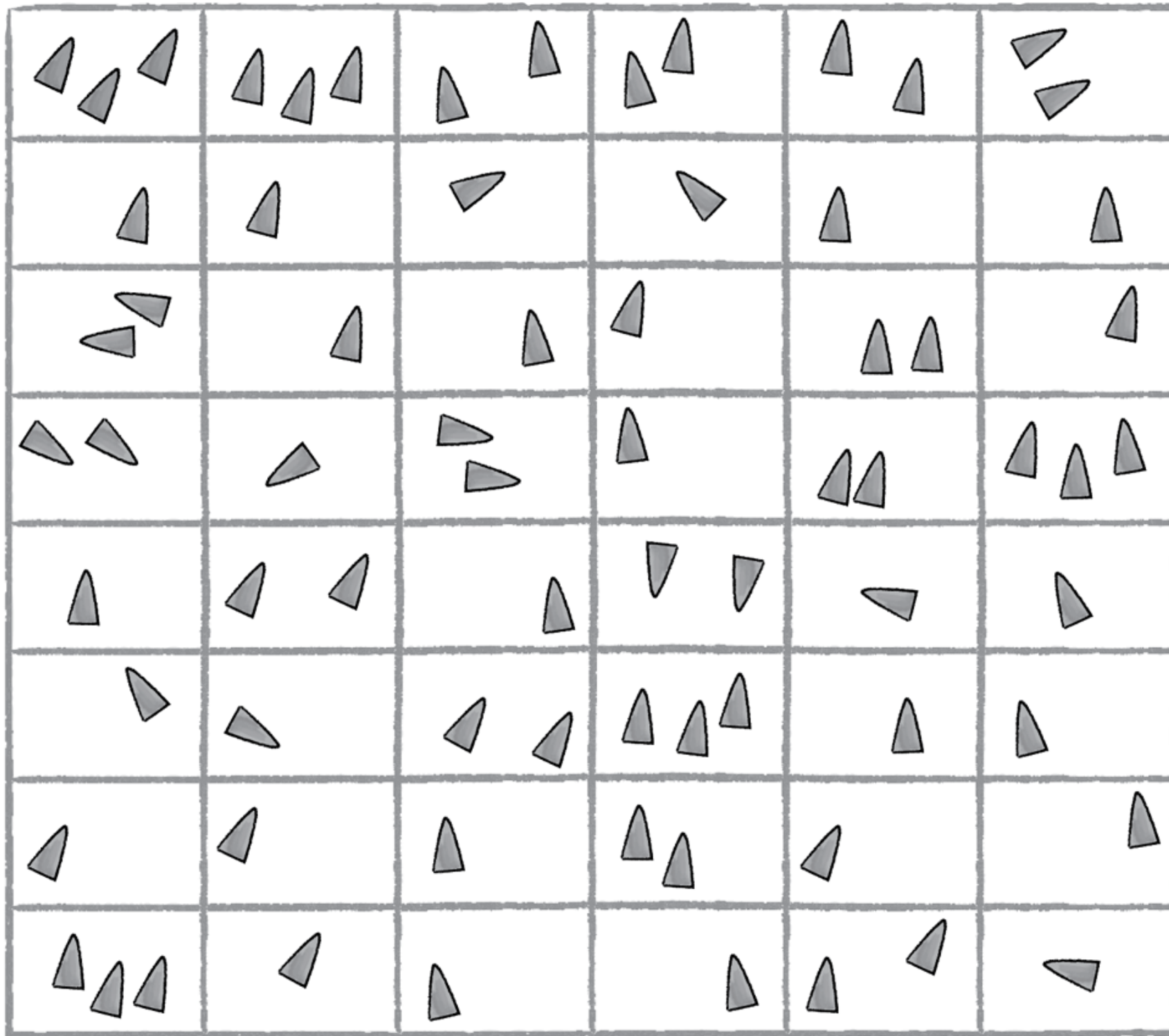
The term ***autonomous agent*** generally refers to an entity that makes personal choices about how to act in its environment without any influence from a leader or global plan.

Agents deal with concept of forces that come from ***within***.

An autonomous agent has a ***limited ability to perceive environment***.

An autonomous agent ***processes*** the information from its environment and ***calculates an action***.

Agents



(from 'The Nature of Code.')

Agents

Important behaviours of Agents.

Action Selection. An agent has a goal (or goals) and can select an action (or a combination of actions) based on that goal. The agent takes a look at its environment and calculates an action based on its observations. Some actions could be: ***seek a target, avoid an obstacle*** or ***follow a path***.

Steering. Once an action has been selected, an agent has to calculate its next move. This is its steering force.

```
Accrete.scd
a = AcGraph(2, 0);

fork {
  a.loadAverse(thisProcess.interpreter, buff
  s.sync;

  g = AcGraph(2, 0);
  g.initRouting(\avarse);

  g.addNodes(a.averseNames.collect {|name| [

  AcGrowth.ring(g, a.averseNames, decayRate:
  AcGrowth.crossLinks(g, 2, decayRate: rrand

  ~t1 = AcTraversal(\fast1, g,
    durFunc: { rrand(1.0, 3.0) }, fadeTime
  ~t2 = AcTraversal(\fast2, g,
    durFunc: { rrand(1.0, 2.5) }, fadeTime
    deactivateOnLeave: false);
  ~t3 = AcTraversal(\mid, g,
    durFunc: { rrand(1.5, 3.0) }, fadeTime
    deactivateOnLeave: false);
  ~t1.walk(\r01);
  ~t2.walk(\s08);
  ~t3.walk(\q01);

  ~evo = AcGrowth.evolve(g);

  g.addProcess(\gain, {|in| in * 1.2 });

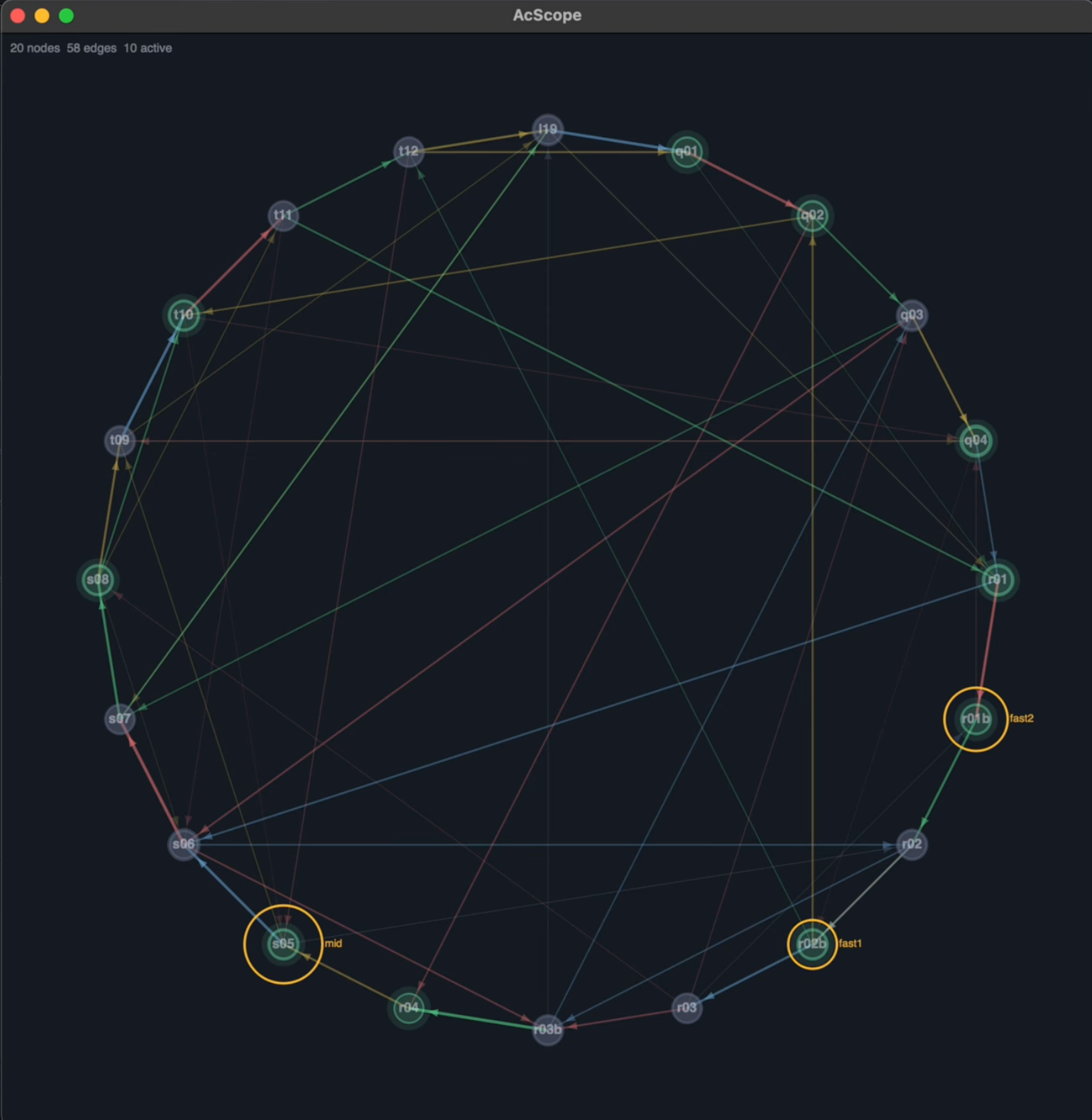
  ~g = g;
  { ~scope = g.scope([~t1, ~t2, ~t3], 900, 9
  g.print;

  };
}
)

~scope.layoutForce(150); ~scope.userView.refresh;

~scope.close; ~evo.stop; ~t1.stop; ~t2.stop; ~t3.s

~/Accrete.scd* 342:31
```



Cellular Automata

Cellular Automata

A CA is an ***abstract machine*** that at any time is in one of a number of possible states.

A ***binary automaton*** may be in either of the two states: on or off.

The state of every automaton in the grid is updated simultaneously in discrete time steps. The future state of a single automaton is governed by its own current state, the current state of all its neighbors, and a set of ***transition rules***.

Cellular Automata

A cellular automaton is a model of a system of “cell” objects with the following characteristics.

- * The cells live on a **grid** . (Often in one and two dimensions, though a cellular automaton can exist in any finite number of dimensions.)
- * Each cell has a **state** . The number of state possibilities is typically finite. The simplest example has the two possibilities of 1 and 0 (otherwise referred to as “on” and “off” or “alive” and “dead”).
- * Each cell has a **neighborhood** . This can be defined in any number of ways, but it is typically a list of adjacent cells.

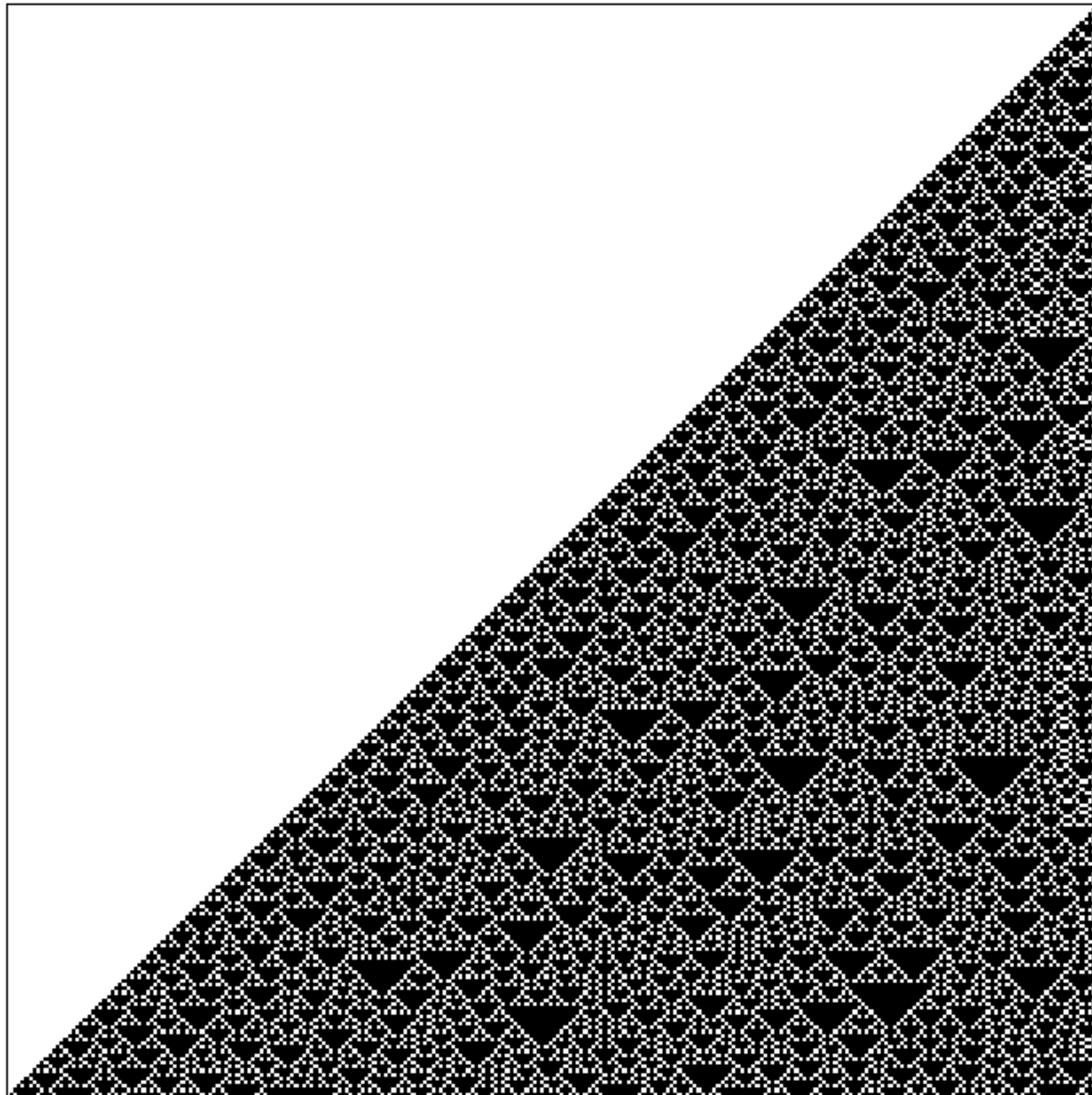
Cellular Automata

Cellular automata systems contain units, or cells, that can be in certain ***states***.

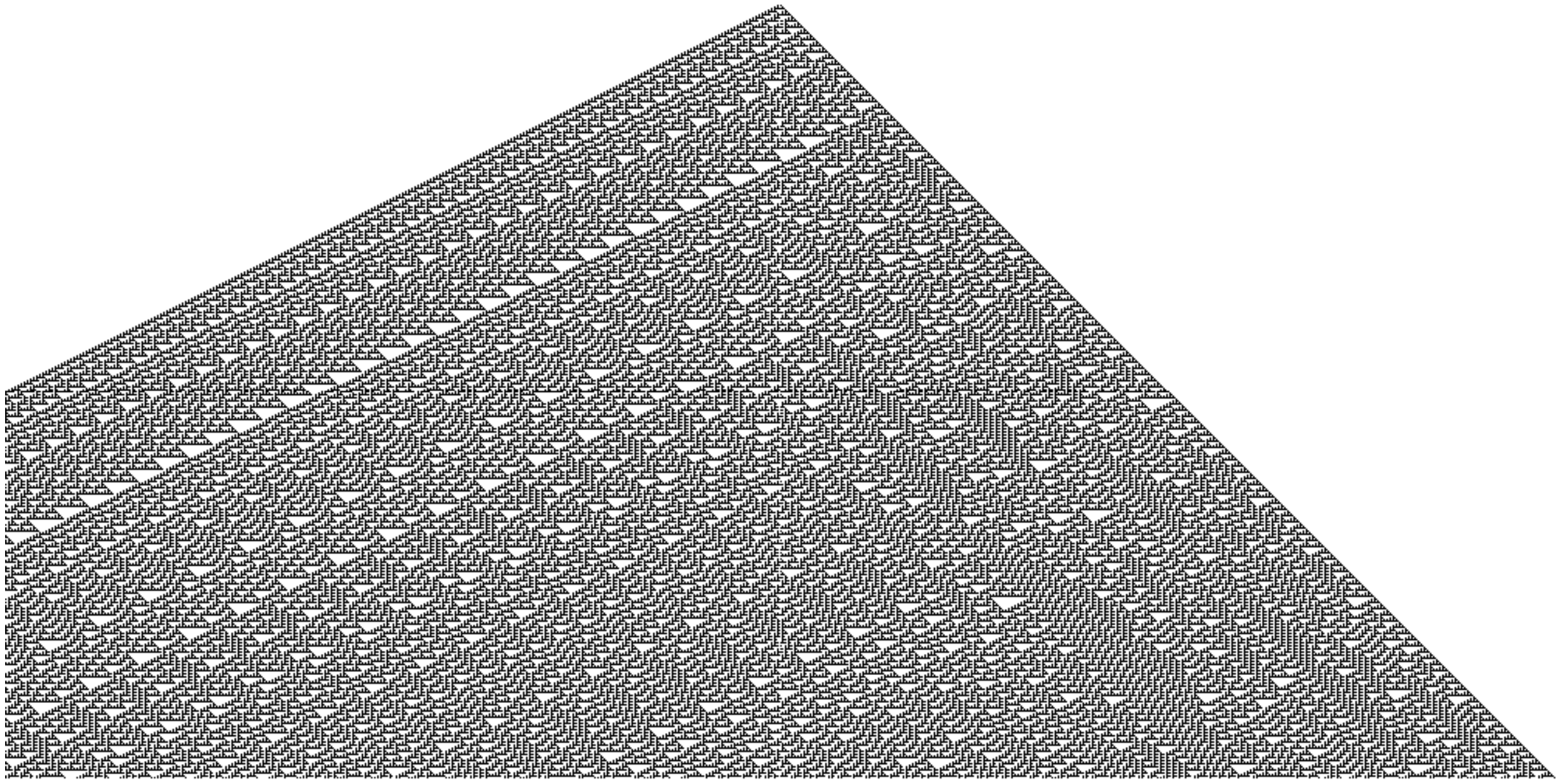
Various ***rules*** are applied to the cells to change their individual states, and thus the state of the system as a whole. Even with a few simple rules, very ***complex behavior*** sometimes results. Perhaps somehow similar to L- systems and chaotic systems.

All of these start with an initial system that is iteratively transformed based on known rules, and can result in very complex fractal results.

Cellular Automata



Cellular Automata



Game of Life

Probably the most common cellular automata system is Conway's ***Game of Life*** , created by the mathematician John Conway.

Based on the initial configuration, various forms can arise. Some can travel, while some are self- sustaining and even self- replicating.

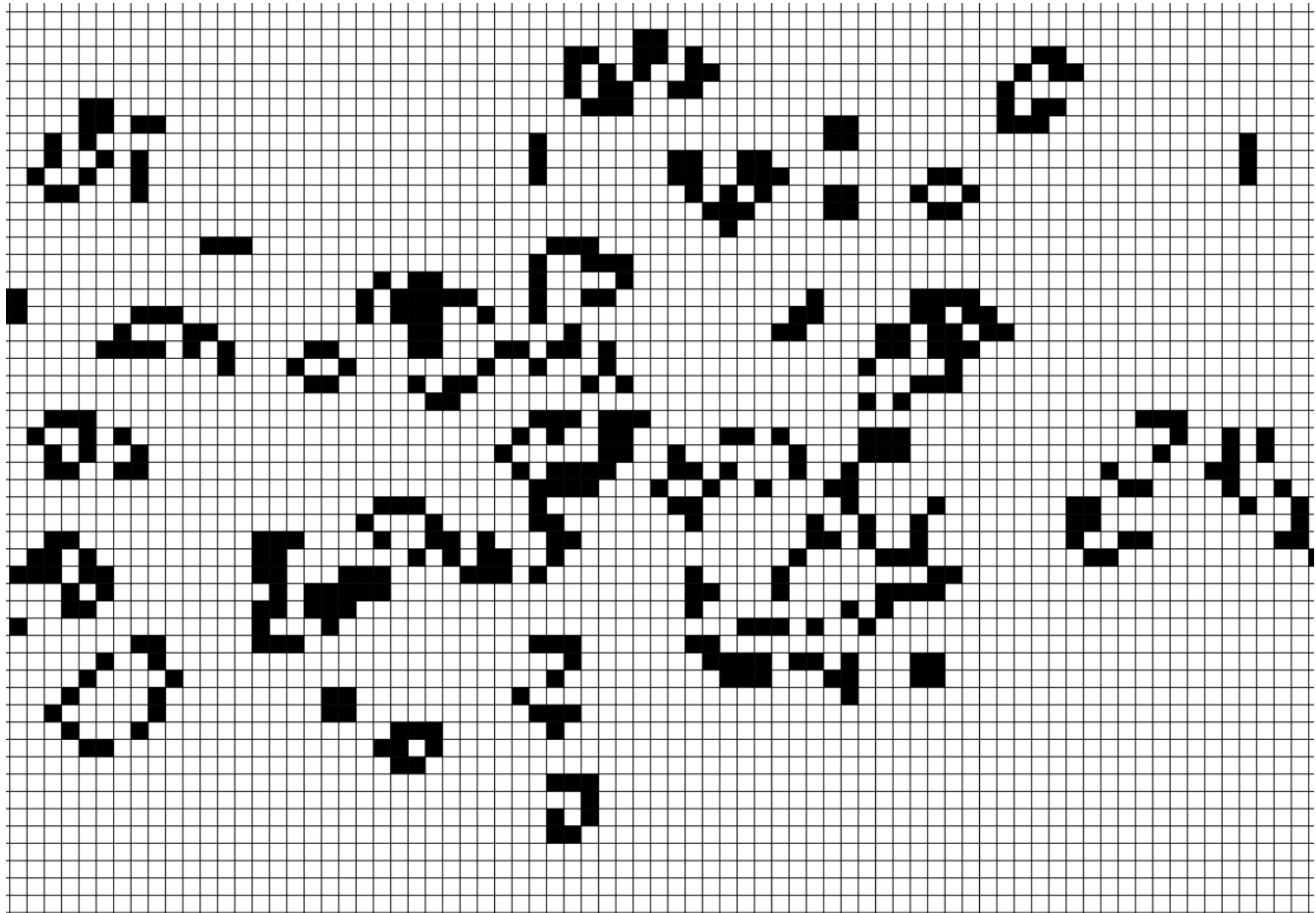
There are even instances of a sort of symbiosis, where different forms work together to sustain themselves and create new forms.

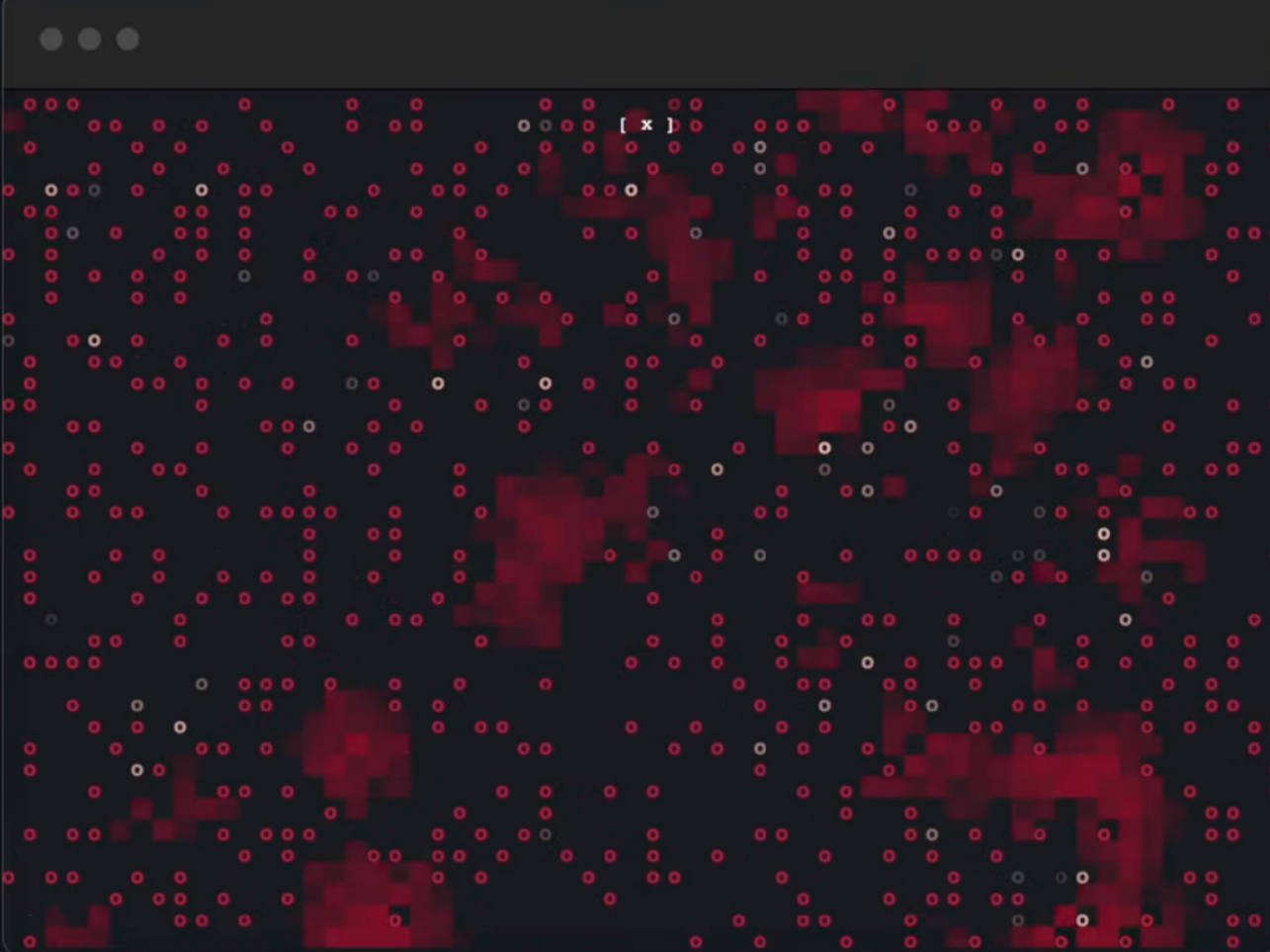
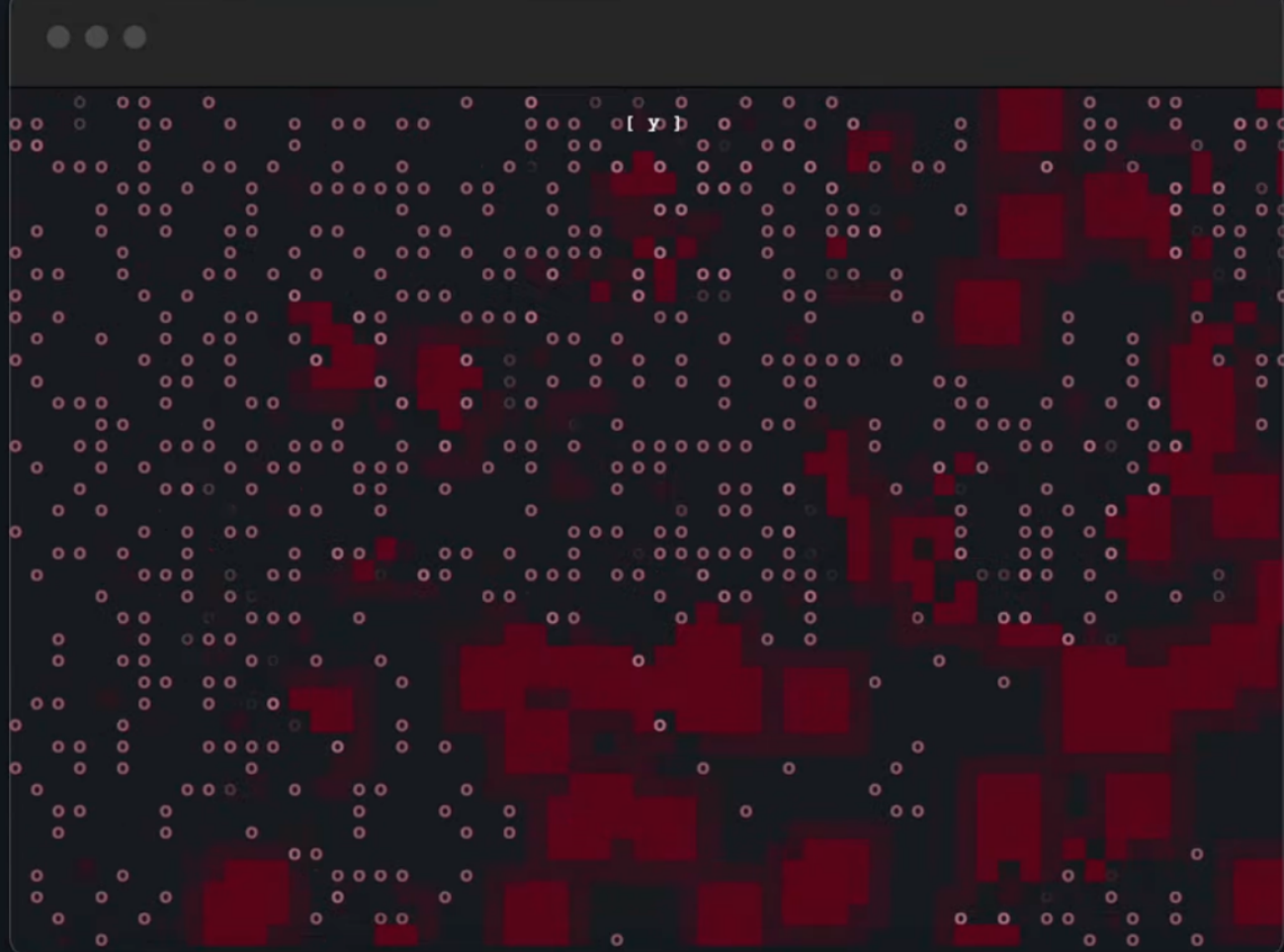
Game of Life

The default rule states that a cell is "born" if it has exactly 3 neighbours. It stays alive if it has 2 or 3 living neighbours, and dies otherwise, is symbolised as "**23/3**".

The first number, or list of numbers, is what is required for a cell to continue. The second is the requirement for birth. Hence "**16/6**" means "a cell is born if there are 6 neighbours, and lives on if there are either 1 or 6 neighbours".

Game of Life





___.scd

```
~sim.attach("x", \pointb, "maxAge", \ffrom, 1900, 2000);

~sim.bind(\pointa)

~sim.bind(\pointb)

~sim.bind(\inv)

(

~sim.focus("x", 9); ~sim.focus("y", 10);
~of.action.interrupt(\pointa); ~of.action.interrupt(\pointb); ~of.action.interrupt(\pointc);

NF(\leaf, { |sdm=1, repeats=2, from=1, to=20, mod=40, amp=0.7, mf=0.03 |
  var sd = SampleDur.ir * sdm;
  HPF.ar(DemandEnvGen.ar(
    Dseq([Dseries(-1.0, Array.rand(4, 0.01, 0.1), 30)], inf),
    Dseq([
      Dwhite(from, to, repeats),
      Dwhite(from/1.5, to*3, repeats)
    ], inf) * sd
  ).clip(SinOsc.ar(Array.rand(4, mod, mod*0.9))) * mf * amp, 50);
}).play;
```

a BrutesAndBullies

a BrutesAndBullies

a BrutesAndBullies

NF('learub')

CheckBadValues: NaN found in Synth 1693, ID 0 (previous 16852385 values were normal)

CheckBadValues: NaN found in Synth 1693, ID 0 (previous 16852385 values were normal)

CheckBadValues: NaN found in Synth 1693, ID 0 (previous 16852385 values were normal)

CheckBadValues: NaN found in Synth 1693, ID 0 (previous 16852385 values were normal)

CheckBadValues: normal found in Synth 1693, ID 0 (previous 96 values were NaN)

CheckBadValues: normal found in Synth 1693, ID 0 (previous 96 values were NaN)

CheckBadValues: normal found in Synth 1693, ID 0 (previous 96 values were NaN)

CheckBadValues: normal found in Synth 1693, ID 0 (previous 96 values were NaN)

~/___.scd* 70:25

LF 1 deprecation UTF-8 SuperCollider

Genetics

Genetics

In a genetic algorithm (GA), the desired output is a ***solution*** to some problem.

The input to the GA has two parts: a ***population*** of candidate programs, and a ***fitness function*** that takes a candidate program and assigns to it a fitness value that measures how well that program works on the desired task.

The basic techniques of the GAs are designed to simulate processes in natural systems necessary for ***evolution***, specially those follow the principles first laid down by Charles Darwin of "*survival of the fittest*".

Genetics

For natural selection to occur the following must be present.

Inheritance. Children receive the properties of their parents where their traits are passed down to their children in the next generation of creatures.

Variation. There must be a variety of traits present in the population. or a means with which to introduce variation.

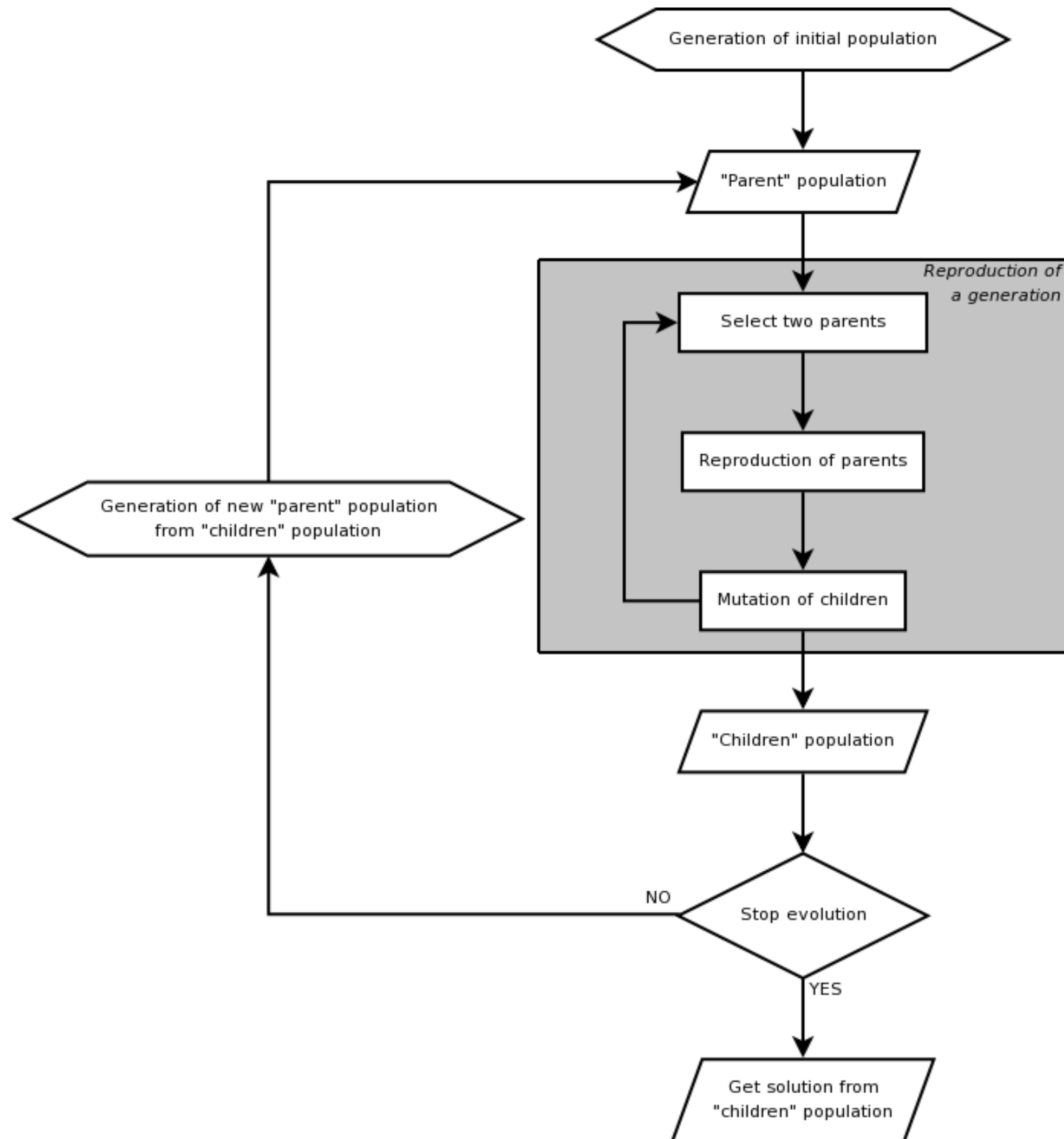
Selection. There must be a mechanism by which some members have childred (and pass down their genetic information and some do not). Typically referred to as “survival of the fittest.”

Genetic Algorithm

1. Generate the *initial population*. X random individuals.
2. Calculate the *fitness* of each individual in the population.
3. Apply *evolution* to the current population of strategies to create a new population:
 - (a) Choose two parent individuals from the current population probabilistically *based on fitness*.
 - (b) **Mate** the two parents to create two children.
 - (c) With a small probability, *mutate* numbers in each child.
 - (d) Put the two children in the *new population*.

Once the new population has 200 individuals, return to step 2 with this new generation.

Genetic Algorithm



Florian Hecker - Stereo F.A.N.N (2010)



Florian Hecker - Stereo F.A.N.N.

Total duration (30 min 4 s).

“The sound generation is created with a customized version of SuperCollider and builds on four particle synthesis / processing techniques including: Glissons, Pulsars, Granulation and Waveset Stretching. These are all controlled via two separate Game-of-Life cellular automata algorithms with circular boundary conditions.”

```

(
  NF(\iop, {|freq=78, mul=1.0, add=0.0|
    var noise = LFNoise1.ar(0.001).range(freq, freq + (freq * 0.1));
    var osc = SinOsc.ar([noise, noise * 1.04, noise * 1.02, noise * 1.08],0,0.2);
    var out = DFm1.ar(osc,freq*4,SinOsc.kr(0.01).range(0.92,1.05),1,0,0.005,0.7);
    HPF.ar(out, 40)
  }).play;
)

(
  NF(\dsc, {|freq = 1080|
    HPF.ar(
      BBandStop.ar(Saw.ar(LFNoise1.ar([19,12]).range(freq,freq*2), 0.2).excess(
        SinOsc.ar( [freq + 6, freq + 4, freq + 2, freq + 8])),
        LFNoise1.ar([12,14,10]).range(100,900),
        SinOsc.ar(20).range(9,11)
      ), 80)
    ).play;
)

var <>pindex, <>cindex;

initialize {
  if(pindex.isNil, { pindex = 1000 });
  if(cindex.isNil, { cindex = 2000 });
}

clearProcessSlots {
  pindex = 1000;
  (this.pindex - 1000).do{|i| this[this.pindex+i] = nil; }
}

clearOrInit {|clear=true|
  if(clear == true, { this.clearProcessSlots() }, { this.initialize() });
}

transform {|process, index|
  if(index.isNil && pindex.isNil, {
    this.initialize();
  });

  pindex = pindex + 1;
  this[pindex] = \filter -> process;
}

control {|process, index|
  var i = index;

  if(i.isNil, {
    this.initialize();
    cindex = cindex + 1;
    i = cindex;
  });

  this[i] = \pset -> process;
}

(
  NF(\depfm, {|freqMin=5, freqMax=20, mul=20, add=80, rate=0.5, modFreq=2100, index=0.3, amp=0.2|
    var trig, seq, freq;
    trig = Dust.kr(rate);
    seq = Diwhite(freqMin, freqMax, inf).midicps;
    freq = Demand.kr(trig, 0, seq);
    HPF.ar(PMOsc.ar(LFCub.kr([freq, freq/2, freq/3, freq/4], 0, mul, add),
      LFNoise1.ar(0.3).range(modFreq,modFreq*2), index) * amp, 50)
  }).play;
)

```

Exercises

Exercises

1. Extend a Cellular Automata example by mapping the array of 0 and 1s to a rhythmic pattern (for example using sine waves with a percussive envelope).
2. Extend a Cellular Automata example by using the arrays of 0 and 1s to control a continuous variable and try to design the unfolding in time so that the clock is not heard as a leading element.
3. Extend the Genetic Algorithm by favouring more difference than similarity. Do so by adjusting the fitness function. Use sound.
4. Extend the Genetic Algorithm by changing both the crossover and mutation function by using increased randomness in search for a solution. Use sound.

Exercises

5. Extend the Boids example by replacing the per-boid voices with a single voice driven by a global property of the flock (its centroid, average velocity, or compactness).
6. Extend the Lorenz example by detecting when the trajectory switches between the two lobes of the butterfly and triggering a percussive event on each crossing.
7. Extend the BouncingBalls example by adding a new per-ball behaviour to the catalogue. Give it its own sound.
8. Extend the TraerPhysics example by sonifying the springs themselves rather than the particles.